

tf.compat.v1.sparse_merge Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/sparse_merge

*Combines a batch of feature ids and values into a single SparseTensor.
(deprecated)*

Function Signatures

```
tf.compat.v1.sparse_merge( sp_ids, sp_values, vocab_size,  
name=None, already_sorted=False )
```

Code Examples

```
tf.compat.v1.sparse_merge(  
    sp_ids, sp_values, vocab_size, name=None, already_sorted=False  
)
```

```
tf.compat.v1.sparse_merge(  
    sp_ids, sp_values, vocab_size, name=None, already_sorted=False  
)
```

```
vector1 = [-3, 0, 0, 0, 0, 0]
vector2 = [ 0, 1, 0, 4, 1, 0]
vector3 = [ 5, 0, 0, 9, 0, 0]
```

```
vector1 = [-3, 0, 0, 0, 0, 0]
vector2 = [ 0, 1, 0, 4, 1, 0]
vector3 = [ 5, 0, 0, 9, 0, 0]
```

```
examples = [Example(features={
    "ids": Feature(int64_list=Int64List(value=[0])),
    "values": Feature(float_list=FloatList(value=[-3]))}),
    Example(features={
    "ids": Feature(int64_list=Int64List(value=[1, 4, 3])),
    "values": Feature(float_list=FloatList(value=[1, 1, 4]))}),
    Example(features={
    "ids": Feature(int64_list=Int64List(value=[0, 3])),
    "values": Feature(float_list=FloatList(value=[5, 9]))})]
```

```
examples = [Example(features={
    "ids": Feature(int64_list=Int64List(value=[0])),
    "values": Feature(float_list=FloatList(value=[-3]))}),
    Example(features={
    "ids": Feature(int64_list=Int64List(value=[1, 4, 3])),
    "values": Feature(float_list=FloatList(value=[1, 1, 4]))}),
    Example(features={
    "ids": Feature(int64_list=Int64List(value=[0, 3])),
    "values": Feature(float_list=FloatList(value=[5, 9]))})]
```

```
SparseTensor(indices=[[0, 0], [1, 1], [1, 3], [1, 4], [2, 0], [2,
3]],
values=[-3, 1, 4, 1, 5, 9],
dense_shape=[3, 6])
```

```
SparseTensor(indices=[[0, 0], [1, 1], [1, 3], [1, 4], [2, 0], [2, 3]],
values=[-3, 1, 4, 1, 5, 9],
dense_shape=[3, 6])
```

Documentation

Combines a batch of feature ids and values into a single SparseTensor.
(deprecated)

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.sparse.merge`

The most common use case for this function occurs when feature ids and their corresponding values are stored in Example protos on disk.

`parse_example` will return a batch of ids and a batch of values, and this function joins them into a single logical SparseTensor for use in functions such as `sparse_tensor_dense_matmul`, `sparse_to_dense`, etc.

The SparseTensor returned by this function has the following properties:

- `indices` is equivalent to `sp_ids.indices` with the last

dimension discarded and replaced with `sp_ids.values`.

- `values` is simply `sp_values.values`.
- If `sp_ids.dense_shape = [D0, D1, ..., Dn, K]`, then

`output.shape = [D0, D1, ..., Dn, vocab_size]`.

For example, consider the following feature vectors:

These might be stored sparsely in the following Example protos by storing only the feature ids (column number if the vectors are treated as a matrix) of the non-zero elements and the corresponding values:

The result of calling `parse_example` on these examples will produce a dictionary with entries for "ids" and "values". Passing those two objects to this function along with `vocab_size=6`, will produce a `SparseTensor` that sparsely represents all three instances. Namely, the `indices` property will contain the coordinates of the non-zero entries in the feature matrix (the first dimension is the row number in the matrix, i.e., the index within the batch, and the second dimension is the column number, i.e., the feature id); `values` will contain the actual values. `shape` will be the shape of the original matrix, i.e., (3, 6). For our example above, the output will be equal to:

This method generalizes to higher-dimensions by simply providing a list for both the `sp_ids` as well as the `vocab_size`.

In this case the resulting `SparseTensor` has the following properties:

- `indices` is equivalent to `sp_ids[0].indices` with the last

dimension discarded and concatenated with

`sp_ids[0].values, sp_ids[1].values, ....`

- `values` is simply `sp_values.values`.
- If `sp_ids.dense_shape = [D0, D1, ..., Dn, K]`, then

`output.shape = [D0, D1, ..., Dn] + vocab_size`.

Returns

`## Raises`